

Построение пайплайнов данных

Оптимизация распределённых алгоритмов

Дмитрий Малахов



Максим Новосельцев

20 февраля 2026 года

Spark: Driver и Executors

Driver (Главный):

- Хранит DAG (план вычислений) — граф зависимостей операций
- Разбивает работу на задачи (Tasks) и отправляет Executor'ам
- Собирает результаты

Уязвимость: Если Driver перегружен данными — приложение падает

Executors (Рабочие лошадки):

- Выполняют задачи в своих JVM-процессах (на разных машинах)
- Хранят часть данных (кэш, промежуточные результаты)
- Отчитываются Driver'у о прогрессе

Уязвимость: Если Executor падает, его задачи перезапускаются на другом

Spark UI — Окно в ваш кластер

Экран	Что смотреть	Признак проблемы
Jobs	Список всех действий (<code>count</code> , <code>save</code> , <code>collect</code>)	Job висит бесконечно
Stages	Длительность задач (Task Timeline)	Один таск час, остальные 5 мин = перекос
Stages	Shuffle Read/Write	Большие серые полосы = дорогой shuffle
Executors	Storage Memory	Мало места для кэша
Executors	GC Time	>5-10% = проблемы с памятью
Executors	Errors	"Container killed" = не хватило памяти

Broadcast Variables — Спасаемся от Shuffle

Проблема: Вы делаете Join RDD (`rdd1.join(rdd2)`).

Spark нужно сгруппировать все записи с одинаковым ключом.

Это вызывает Shuffle (запись на диск, передача по сети) — медленно и дорого.

Идея: Если `rdd2` маленький (справочник стран, курс валют), зачем гонять его туда-сюда?

Решение:

```
# Маленький справочник
mapping = {"a": 1, "b": 2, "c": 3}
bc = sc.broadcast(mapping)

# Большой RDD
rdd = sc.parallelize(["a", "b", "c", "d", "a", "b"])
result = rdd.map(lambda x: bc.value.get(x, 0))
print(result.collect()) # [1, 2, 3, 0, 1, 2]
```

Broadcast: Техника безопасности

Плюсы:

- Ускорение join'ов в 10-100 раз
- Снижение нагрузки на сеть
- Простота использования

Когда использовать:

- Таблицы-справочники (до ~100MB)
- Списки стоп-слов для фильтрации
- Параметры ML-моделей (веса, коэффициенты)
- Конфигурационные файлы с правилами

Минусы и ограничения:

- Данные должны помещаться в память Executor'а. Если вы разошлете 5GB справочник, каждый Executor упадет с OOM (Out of Memory)
- Read-only. Нельзя менять переменную внутри тар (изменения будут видны только локально, не влияя на другие партиции)
- Сериализация. Переменная копируется целиком, даже если вам нужна только её часть

Практическое правило:

Если данные < 100MB и read-only — broadcast!

Аккумуляторы — Сбор метрик на лету

Проблема: Внутри map или filter вы не можете просто увеличить счетчик (counter += 1) — он будет увеличиваться только локально на Executor'e, а Driver об этом не узнает.

Когда использовать:

- Подсчет битых/пропущенных записей
- Подсчет количества обработанных объектов
- Валидация данных (сколько значений вне допустимого диапазона)
- Мониторинг прогресса длительных операций

Решение:

```
# Создаем аккумулятор на Driver'e
null_counter = sc.accumulator(0)
warning_counter = sc.accumulator(0)

def process_row(row):
    if row is None:
        null_counter.add(1) # Безопасно увеличиваем
        return "EMPTY"
    if len(row) > 1000:
        warning_counter.add(1) # Слишком длинная строка
    return row.lower()

# Применяем
processed = rdd.map(process_row)
processed.count() # Действие запускает вычисления

print(f"Найдено null-строк: {null_counter.value}")
print(f"Найдено длинных строк: {warning_counter.value}")
```

Аккумуляторы: Опасность больших данных

Почему нельзя собирать все ошибки в список?

Важнейшее ограничение:

- Аккумуляторы передают данные от Executor к Driver
- Driver хранит **финальное значение** в своей памяти

Уязвимость: Если Driver перегружен данными — приложение падает

Правило: Используйте аккумуляторы для **агрегированных метрик** (счетчики, суммы), а не для **коллекций данных**. Максимум можно собрать пару сотен килобайт, но на практике лучше ограничиться десятками байт.

Исключение: в ряде случаев можно собрать ограниченное количество примеров чего-либо, примеры хранятся в коллекции, но при этом не требуют много памяти.

Кастомные аккумуляторы — Когда мало посчитать

Стандартные аккумуляторы умеют только складывать числа (Long, Double).

А если нам нужно собрать более сложную метрику?

Почему это безопасно:

- Каждый Executor хранит свое локальное множество ошибок
- На Driver передается только результат объединения множеств
- Если ошибок 10 миллионов, но типов ошибок 10 — на Driver уйдет только 10 строк

```
from pyspark import AccumulatorParam

class UniqueErrorsAccumulator(AccumulatorParam):
    """Собирает только уникальные сообщения об ошибках"""

    def zero(self, initial_value):
        # Начальное значение для каждого Executor'a
        return set()

    def addInPlace(self, set1, set2):
        # Объединяем два множества (локально на Executor'e)
        if isinstance(set2, set):
            return set1.union(set2)
        else:
            # Добавляем одиночную ошибку
            set1.add(set2)
            return set1
```


Когда аккумуляторы врут?

Контекст:

Spark — отказоустойчивая система. Если Executor упал, его задачи запускаются на другом узле.

Проблема:

Аккумуляторы не знают о перезапусках.

Важно запомнить:

- Сбои – слабое место аккумуляторов
- Используйте их для трендов и метрик, а не для финансовой отчетности
- Для точных подсчетов используйте операции RDD (count, reduce)

```
# НЕПРАВИЛЬНО (счетчик зависится)
counter = sc.accumulator(0)

def process_partition(iterator):
    for row in iterator:
        counter.add(1) # Считаем каждую строку
        yield process(row)

# Сценарий:
# 1. Executor 1 начал обрабатывать партицию 1, насчитал 1000 строк
# 2. Executor 1 упал
# 3. Spark перезапускает партицию 1 на Executor 2
# 4. Executor 2 снова считает те же 1000 строк
# 5. Итог: counter.value = 2000, хотя реально строк 1000
```

Партиционирование — Основа параллелизма

RDD разрезан на партии. Одна партия = одна задача (Task) = одно ядро.

Проблемы:

- Слишком мало партий (< кол-ва ядер): Ядра простаивают
- Слишком много партий: Тысячи мелких задач грузят планировщик

Управление партиями:

Операция	Shuffle	Когда использовать
<code>repartition(n)</code>	ДА	Увеличить параллелизм или выровнять данные
<code>coalesce(n)</code>	НЕТ	После фильтрации — уменьшить партии без shuffle
<code>partitionBy(p)</code>	ДА	Перед серией groupBy/join по одному ключу
<code>repartitionAndSortWithinPartitions(p)</code>	ДА	Когда нужны сортированные данные внутри партий (topN, merge join)

Сериализация — Критический фактор скорости

Когда Spark мешает данные или отправляет задачи, он их сериализует.

Типы сериализации:

Тип	Скорость	Размер	Когда использовать
Java Serialization	Медленная	Большой	По умолчанию
Kryo Serialization	Быстрая (x10)	Малый	Для кастомных классов
Pickle (PySpark)	Средняя	Средний	Для Python-объектов

Эффект: Особенно заметен, если ваши RDD содержат сложные объекты (не просто числа и строки) — ускорение в 2-10 раз.

Модель памяти Spark

```
executor.memory = 8g
```

```
RESERVED: 300 MB (фиксировано)
```

```
USER MEMORY: (mem-300MB)*(1 - spark.memory.fraction) → UDF, код (25%)
```

```
SPARK MEMORY: (mem-300MB)*spark.memory.fraction (75% при fraction=0.75)
```

```
└─ EXECUTION: Spark*(1 - spark.memory.storageFraction) → shuffle, join (50%)
```

```
└─ STORAGE: Spark*spark.memory.storageFraction → cache (50%)
```

```
spark.executor.memoryOverhead: память вне кучи (стеки, PySpark)
```

```
→ 10-15% от executor.memory, минимум 384MB
```

Shuffle Spill — Когда памяти не хватает

Контекст: При выполнении операций, требующих сортировки (**а какие требуют?**), Spark пытается держать данные в памяти.

Проблема: Если на одну партицию попадает слишком много данных (**перекос**), памяти может не хватить.

Механизм Spill (Сброс на диск):

- Spark начинает собирать данные для ключа в памяти
- Если память заканчивается, Spark сбрасывает часть данных на диск
- Потом читает их с диска и дособирует

Цена: Операции с диском в 100-1000 раз медленнее операций с памятью.

Shuffle Spill: Контроль точки сброса

Параметр: `spark.shuffle.spill.numElementsForceSpillThreshold`

Суть: Количество элементов в одной партии, после которого Spark принудительно начинает сбрасывать данные на диск, даже если память еще есть.

Значение по умолчанию: `Long.MAX_VALUE` (отключено — сброс только при нехватке памяти)

Зачем это нужно:

- Предотвращает долгие GC паузы при очень больших коллекциях в памяти
- Делает поведение более предсказуемым
- Позволяет избежать OOM (если объекты могут быть большими и Spark не может корректно оценить возможности хранения).

Ключевые проблемы

- **Слишком много объектов** → долгий GC, дополнительные накладные расходы
- **Выделили мало памяти на ядро** → долгий GC, риски падения по **Out Of Memory** (может возникнуть и во время udf и во время shuffle и во время cache)
- **Выделили много памяти на ядро** → часть ядер простаивает
- **Несбалансировали партии** → все задачи отработали, одна дорабатывает больше N% времени стадии.
- **Чрезмерное количество партиций** → тормозит Spark UI, дополнительные накладные расходы
- **Необоснованное использование диска, повторные вычисления и неоптимальное количество стадий** → меньше скорость обработки

Все это приводит к неэффективному использованию ресурсов кластера

Итоги

1. При настройке памяти и количестве стадий следует отталкиваться от минимальных значений
2. Если ничего не упало, то необходимо проверить наличие Spill и GC Time в Spark UI
 1. Если значения приемлемые, больше ничего не нужно.
 2. Если GC Time слишком большой, нужно подумать над сокращением количества объектов, иначе постепенно увеличивать память для UDF
 3. Если Spill слишком большой, то стоит попробовать сбалансировать партиции или увеличить память на Storage или Execution
3. Если произошло OOME, то аналогичные действия:
 1. Как если бы GC Time был слишком большим (если проблема вероятно в UDF)
 2. Как если бы Spill слишком большой (если проблема не в UDF), также можно регулировать настройки Spill.

Домашнее задание

Провести эксперименты с оптимизацией Spark-приложений:

- Сгенерировать датасет (можно взять готовый)
- Обозначить проблемы, которые мешают эффективной обработке
- Продемонстрировать несколько оптимизаций

Чем более интересными получатся кейсы, тем лучше

*** Решить проблему, которая не была описана выше**